

The Price of a Proof: A Controlled Token-Cost Comparison of Formal Contract-Driven Development and Ordinary Development-plus-Review

Withheld for review

Study repository and full data available; see Data Availability

Abstract. Formal methods are defended on correctness and attacked on cost, yet the cost claim is seldom measured against a realistic alternative doing the same work. We ran a controlled comparison on a 7,343-line Z80 assembly chess engine into which an independent agent seeded seven single-byte faults, each with a sealed acceptance test written before any repair was attempted. Two arms fixed every fault: ordinary development-plus-review, and Formal Contract-Driven Development (FCDD) working from a pre-built machine-checked contract. Across seven matched pairs FCDD was more expensive on 7 of 7 (exact two-sided sign test $p = 0.016$), median ratio $3.40\times$; its upfront contract never amortised. Replicating the pair whose arms were closest revealed that the ordinary arm is *bimodal* — it produced either a minimal one-byte repair (\$8.66, \$11.78) or a redesign (\$44.27, \$41.35) — while FCDD produced the minimal repair in all three runs. Matched on solution, FCDD costs a median $5.0\times$ more. Blinded, order- counterbalanced grading of 14 judgements favoured FCDD on 6 of 7 pairs ($p = 0.125$), concentrated in test quality. We report three measurement findings we regard as more transferable than the ratio: an instruction to review “until no findings remain” has no fixed point; measured cost covaries with review-loop length; and processes killed by infrastructure emit no cost record.

Keywords: Formal methods · Empirical software engineering · Cost measurement · LLM agents · Code review

1 Introduction

The case for formal methods is argued on one axis and rebutted on another. Proponents show what proof buys; opponents answer that it costs too much. The disagreement persists partly because the cost side is rarely measured against a competent alternative performing the identical repair on the identical defect, judged by the identical gate.

This study measures it, and its more useful contribution turned out to be how badly that measurement can go wrong. Five times a confident, plausible finding proved to be an artefact of our own apparatus rather than a property of

the methods; three were caught by our own mechanical checks and two only by adversarial review of a finished draft. Each correction moved the headline, and two reversed its direction.

2 Related Work

Empirical cost of formal methods. The field’s own guidance notes that empirical studies of formal methods are rare [5]. What exists is largely retrospective single-project accounting with no control arm: seL4 required 2.2 person-years to build and over 20 to verify [26,15]; ShardStore’s authors compare their overhead to figures *drawn from other papers* [8]; the AWS TLA+ reports rest on counterfactual testimony [18]. The largest survey concedes the gap outright: in a majority of projects surveyed there is no data available on relative time and costs [28]. The classic negative industrial result is observational and unpriced [20].

The one controlled precedent. Sobel and Clarkson ran a controlled formal-versus-informal experiment with twenty student teams [25]; we therefore do not claim the first controlled comparison. They measured product quality and never priced either arm, and Berry and Tichy argued in print that the design could not support even the quality claim [7]. Our sealed acceptance tests and order-counterbalanced blinded grading respond directly to that critique. The closest work in unit of analysis reports 87 minutes to develop each of 73 unit proofs [2], but against a detection-rate baseline rather than a competing method, and in human minutes rather than tokens.

Agentic repair and its cost. LLM agents repairing real defects is established [11], as is the finding that simpler structured pipelines can beat autonomous agents on quality and dollars alike [29]. Our contribution is therefore not that structure helps but *which* structure, at what price. Cost-aware agent evaluation is likewise prior art [13], and per-task token consumption has been characterised in detail, including the finding that runs of the same task can differ by up to $30\times$ [4] — a noise floor any cost delta must clear. Benchmark contamination motivated seeding our own faults [1]. The nearest neighbour varies one prompt line controlling specification grounding for +38 percentage points of correct code [9], but reports no cost accounting.

Review effectiveness — our instrument findings are replication. That review surfaces far more maintainability issues than defects is long established — 75% evolvability to 25% functional [17], replicated in open source with 7–35% of comments discarded [6]. Porter et al. already quantified the signal-to-noise ratio we observed [21], and Rigby and Bird state from industrial data that more reviewers yield “only a minimal increase in the number of comments” and “no increase in the number of change sets submitted” [22]. Our 85 findings without a defect in the fix is therefore a replication. What appears unclaimed is the *stopping rule*: over 80% of human changes involve at most one review iteration [23], so twelve rounds under an unbounded instruction is an order-of-magnitude departure.

Seeded faults and grading: threats we inherit. Mutants couple to 73% of real faults but not to 17% [12]; correlation with real-fault detection weakens once suite size is controlled [19]; hand-seeded faults tend to be harder to detect than real ones [3]. Our defence is that single-statement bugs are an abundant measured class [14]. Position bias in LLM judges was reported by Zheng et al. [30], demonstrated as rank-hacking by reordering [27], and shown strongest when the quality gap is small [24] — exactly our regime. Our first grading round reproduced this failure and we applied the established remedy; that is a confession, not a finding. For method comparison we follow standard guidance [16], and adopt the pair-programming meta-analysis framing of reporting effort as a separate dependent variable rather than equalising it [10].

What remains ours. Searching for prior work pricing a formal-methods workflow in LLM token cost against a conventional one returned nothing we could verify; we state that as a searched-and-not-found result. The defensible contribution is the conjunction: a concurrently-run control arm on the same faults, a directly metered cost unit, a repeated-measures design, pre-committed outcome measurement, and the stopping-rule and bimodality results.

3 Method

An independent agent seeded seven single-fault variants into the engine, each differing from pristine by exactly one byte in the assembled binary, wrote symptom-level reports with no location hints, and sealed an answer key whose hash was published before any repair began. Each $\text{arm} \times \text{defect}$ cell ran as an isolated headless session on the same model at the same effort. FCDD’s contract was built once as a separately measured phase (1,271 lines of specification, 95 machine-checked theorems, empty axiom profile) at \$21.75.

Cost is reported three ways — the platform figure, the same tokens priced with our pre-registered rate table, and raw token totals — because they disagree. Token totals come from each session’s `modelUsage`, which includes reviewer sub-agents; an earlier ledger recorded main-loop usage only and under-counted by 29–99%.

Disclosed asymmetry. FCDD’s contract was built by reading the *pristine* engine, so Arm B held an executable model of correct behaviour covering all seven faults by construction; Arm A received only the symptom report. This is the most consequential design flaw and would not transfer to faults of omission.

4 Instrument findings

Unbounded review has no fixed point. In one cell Arm A fixed the seeded fault correctly on its first pass, then ran 12 review rounds producing 85 findings, none of which was a defect in the fix (OLS slope +0.025 findings/round, $R^2 = 0.001$). Bounding the rule — a round finding no defect closes the gate, cap three

Table 1. Cost per defect, final instrument. † imputed from recovered tokens.

Defect	Arm A	Arm B	Ratio
bug01 quiescence	\$44.27	\$49.63	1.12×
bug02 fifty-move	\$7.32	\$50.37	6.88×
bug03 promotion	\$11.96	\$23.98†	2.01×
bug04 eval sign	\$11.30	\$55.40	4.90×
bug05 castling	\$8.88	\$31.05	3.50×
bug06 TT bound	\$16.44	\$48.69	2.96×
bug07 mate detect	\$7.11	\$24.19	3.40×
Median ratio (platform / rates / tokens) 3.40× / 3.28× / 4.74×			

rounds — cut the same cell from at least \$128.62 with no completion to \$7.32 with a passing gate.

Cost covaries with review effort. Over all 13 final-instrument cells, cost = $4.13 + 21.35 \cdot \text{ArmB} + 7.13 \cdot \text{reviewers}$, $R^2 = 0.826$; controlling for reviewer count attenuates the arm gap from \$30.34 to \$21.35.

Killed processes emit no cost record. One cell reported \$1.06; its first attempt had been killed without emitting a result record. Priced from recovered transcripts at our pre-registered rates the true figure is \$23.98 — an under-count of roughly 20×

5 Results

FCDD was dearer on 7 of 7 pairs (sign test $p = 0.016$), the only statistically supportable cost result here (Table 1, Fig. 1). No crossover exists: N is undefined when the saving is negative on every pair.

Replication and bimodality. Replicating bug01 — required by our pre-registered rule for pairs whose arms differ by under 30% — showed Arm A is *bimodal*: two runs produced a minimal one-byte repair (\$8.66, \$11.78, pristine binary) and two produced a redesign of the quiescence policy (\$44.27, \$41.35). Arm B produced the minimal repair in all three runs (\$49.63, \$79.28, \$51.40). Matched on solution, FCDD costs a median 5.0× (range 4.2–9.2). This is distinct from stochastic token variance [4]: it is strategy selection, and the expensive Arm A branch is the one graders scored at or below the gate floor — which the sealed acceptance script passed regardless.

Quality. All submissions cleared the pre-registered threshold. After the first grading round proved order-confounded, we re-graded all seven pairs in both presentation orders with 14 independent graders: position came out exactly 7/7, every pair was order-consistent, and FCDD was favoured on 6 of 7 ($p = 0.125$). Axis means (A / B): correctness 4.36 / 4.50; clarity 4.64 / 4.29; test quality 4.14 / 4.86. On three defects the two source fixes were byte-identical, so the verdict turned entirely on verification: FCDD produced better *evidence*, not better fixes.

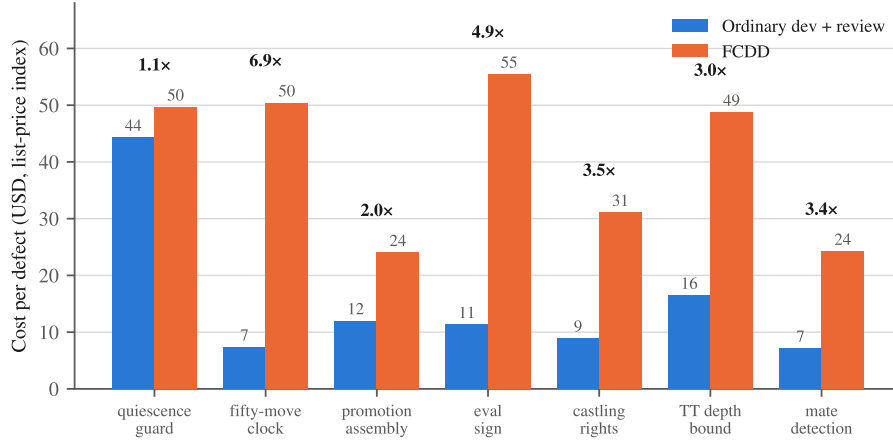


Fig. 1. Cost per defect. FCDD is more expensive on every pair.

6 Threats to Validity

Seven defects, one codebase, one model. The oracle asymmetry above is the most consequential flaw. The instrument changed four times; only final-version cells are compared. Five of seven Arm B and two of seven Arm A cells required a resume after infrastructure interruptions, and summed cost therefore overstates FCDD. The bounded rule caps review rounds, not invocations, granting Arm B 1.8 $\times$ the reviewer invocations. Acceptance scripts were written by the fault-injecting agent, so a shared misconception is undetectable. Grading is label-blind, not treatment-blind. The authors evaluate their own method; every withdrawn claim is published rather than deleted.

7 Conclusions

Both methods reliably repaired real assembly defects. FCDD was more expensive on 7 of 7 matched pairs and never amortised; matched on solution the gap widens to 5.0 $\times$. We make no claim about cost predictability. Quality was close, with FCDD ahead on test quality. Most transferably: review-loop termination dominates measured cost, self-reported costs must be reconciled against transcripts, and grading order must be counterbalanced — we failed each of these at least once, and each failure produced a publishable-looking result that was false.

Acknowledgments. None.

Disclosure of Interests. The method under test was authored by the same human-agent pair conducting this evaluation. Mitigations: pre-registration, independently seeded

faults with a sealed key, mechanical gates written before repair, blinded counterbalanced grading, adversarial review against raw data, and publication of every withdrawn claim. External replication is warranted.

References

1. Aleithan, R., Xue, H., Mohajer, M.M., Nnorom, E., Uddin, G., Wang, S.: SWE-Bench+: Enhanced coding benchmark for LLMs. arXiv preprint arXiv:2410.06992 (2024)
2. Amusuo, P.C., Cochell, O., Le Lievre, R., Patil, P.V., Machiry, A., Davis, J.C.: Do unit proofs work? an empirical study of compositional bounded model checking for memory safety verification. arXiv preprint arXiv:2503.13762 (2025). <https://doi.org/10.48550/arXiv.2503.13762>
3. Andrews, J.H., Briand, L.C., Labiche, Y.: Is mutation an appropriate tool for testing experiments? In: Proc. 27th Int. Conf. on Software Engineering (ICSE). pp. 402–411 (2005). <https://doi.org/10.1145/1062455.1062530>
4. Bai, L., Huang, Z., Wang, X., Sun, J., Mihalcea, R., Brynjolfsson, E., Pentland, A., Pei, J.: How do AI agents spend your money? analyzing and predicting token consumption in agentic coding tasks. arXiv preprint arXiv:2604.22750 (2026)
5. ter Beek, M.H., Ferrari, A.: Empirical formal methods: Guidelines for performing empirical studies on formal methods. *Software* **1**(4), 381–416 (2022). <https://doi.org/10.3390/software1040017>
6. Beller, M., Bacchelli, A., Zaidman, A., Juergens, E.: Modern code reviews in open-source projects: Which problems do they fix? In: Proc. 11th Working Conf. on Mining Software Repositories (MSR). pp. 202–211 (2014). <https://doi.org/10.1145/2597073.2597082>
7. Berry, D.M., Tichy, W.F.: Comments on “formal methods application: An empirical tale of software development”. *IEEE Transactions on Software Engineering* **29**(6), 567–571 (2003). <https://doi.org/10.1109/TSE.2003.1205183>
8. Bornholt, J., Joshi, R., Astrauskas, V., Cully, B., Kragl, B., Markle, S., Sauri, K., Schleit, D., Slatton, G., Tasiran, S., Van Geffen, J., Warfield, A.: Using lightweight formal methods to validate a key-value storage node in Amazon S3. In: Proc. 28th ACM Symp. on Operating Systems Principles (SOSP). pp. 836–850 (2021). <https://doi.org/10.1145/3477132.3483540>
9. Haeri, H., Ghelichi, A.: Specification grounding drives test effectiveness for LLM code. arXiv preprint arXiv:2607.06636 (2026)
10. Hannay, J.E., Dybå, T., Arisholm, E., Sjøberg, D.I.K.: The effectiveness of pair programming: A meta-analysis. *Information and Software Technology* **51**(7), 1110–1122 (2009). <https://doi.org/10.1016/j.infsof.2009.02.001>
11. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: SWE-bench: Can language models resolve real-world GitHub issues? In: Proc. 12th Int. Conf. on Learning Representations (ICLR) (2024)
12. Just, R., Jalali, D., Inozemtseva, L., Ernst, M.D., Holmes, R., Fraser, G.: Are mutants a valid substitute for real faults in software testing? In: Proc. 22nd ACM SIGSOFT Int. Symp. on Foundations of Software Engineering (FSE). pp. 654–665 (2014). <https://doi.org/10.1145/2635868.2635929>
13. Kapoor, S., Stroebel, B., Siegel, Z.S., Nadgir, N., Narayanan, A.: AI agents that matter. arXiv preprint arXiv:2407.01502 (2024)

14. Karampatsis, R.M., Sutton, C.: How often do single-statement bugs occur? the ManySStuBs4J dataset. In: Proc. 17th Int. Conf. on Mining Software Repositories (MSR) (2020). <https://doi.org/10.1145/3379597.3387491>
15. Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H., Winwood, S.: seL4: Formal verification of an OS kernel. In: Proc. 22nd ACM SIGOPS Symp. on Operating Systems Principles (SOSP). pp. 207–220 (2009). <https://doi.org/10.1145/1629575.1629596>
16. Ko, A.J., LaToza, T.D., Burnett, M.M.: A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Software Engineering* **20**(1), 110–141 (2015). <https://doi.org/10.1007/s10664-013-9279-3>
17. Mäntylä, M.V., Lassenius, C.: What types of defects are really discovered in code reviews? *IEEE Transactions on Software Engineering* **35**(3), 430–448 (2009). <https://doi.org/10.1109/TSE.2008.71>
18. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardouff, M.: How Amazon Web Services uses formal methods. *Communications of the ACM* **58**(4), 66–73 (2015). <https://doi.org/10.1145/2699417>
19. Papadakis, M., Shin, D., Yoo, S., Bae, D.H.: Are mutation scores correlated with real fault detection? In: Proc. 40th Int. Conf. on Software Engineering (ICSE). pp. 537–548 (2018). <https://doi.org/10.1145/3180155.3180183>
20. Pfleeger, S.L., Hatton, L.: Investigating the influence of formal methods. *IEEE Computer* **30**(2), 33–43 (1997). <https://doi.org/10.1109/2.566148>
21. Porter, A., Siy, H., Mockus, A., Votta, L.: Understanding the sources of variation in software inspections. *ACM Transactions on Software Engineering and Methodology* **7**(1), 41–79 (1998). <https://doi.org/10.1145/268411.268421>
22. Rigby, P.C., Bird, C.: Convergent contemporary software peer review practices. In: Proc. 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE). pp. 202–212 (2013). <https://doi.org/10.1145/2491411.2491444>
23. Sadowski, C., Söderberg, E., Church, L., Sipko, M., Bacchelli, A.: Modern code review: A case study at Google. In: Proc. 40th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP) (2018). <https://doi.org/10.1145/3183519.3183525>
24. Shi, L., Ma, C., Liang, W., Diao, X., Ma, W., Vosoughi, S.: Judging the judges: A systematic study of position bias in LLM-as-a-judge. *arXiv preprint arXiv:2406.07791* (2025)
25. Sobel, A.E.K., Clarkson, M.R.: Formal methods application: An empirical tale of software development. *IEEE Transactions on Software Engineering* **28**(3), 308–320 (2002). <https://doi.org/10.1109/32.991322>
26. Staples, M., Jeffery, R., Andronick, J., Murray, T., Klein, G., Kolanski, R.: Productivity for proof engineering. In: Proc. 8th ACM/IEEE Int. Symp. on Empirical Software Engineering and Measurement (ESEM) (2014). <https://doi.org/10.1145/2652524.2652551>
27. Wang, P., Lei, L., Chen, L., Cai, Z., Zhu, D., Lin, B., Cao, Y., Kong, L., Liu, Q., Liu, T., Sui, Z.: Large language models are not fair evaluators. In: Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL). pp. 9440–9450 (2024). <https://doi.org/10.18653/v1/2024.acl-long.511>
28. Woodcock, J., Larsen, P.G., Bicarregui, J., Fitzgerald, J.: Formal methods: Practice and experience. *ACM Computing Surveys* **41**(4), 1–36 (2009). <https://doi.org/10.1145/1592434.1592436>

29. Xia, C.S., Deng, Y., Dunn, S., Zhang, L.: Agentless: Demystifying LLM-based software engineering agents. In: Proc. ACM on Software Engineering (FSE) (2025). <https://doi.org/10.1145/3715754>
30. Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I.: Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In: Advances in Neural Information Processing Systems 36 (NeurIPS Datasets and Benchmarks) (2023)